

NUS
National University of Singapore

CS3103 Computer Network Practice
AY26/27 Semester 1

Assignment 4
Group 21

CHEN JUNYAO (A0273316H)
CHONG JIA CHENG CHRISTOPHER (A0273461E)
JANSEN KEN PEGRASIO (A0305755B)
LIM KAI WEI (A0273249X)

National University of Singapore, 21 Lower Kent Ridge Rd, University Hall, 119077, Singapore

1 System Architecture & Protocol Mechanics

LiveRecall employs a single-authoritative-server topology coordinating three distinct network entities (see Figure A1): the speech transcription upstream source (`caption_source.py`), the distributed student clients (browser-based `client/app.js`), and the centralized asynchronous core (`solution_server.py`).

Wire Protocol Framing

The protocol uses lightweight JSON payloads over WebSocket text frames, with the message exchange illustrated in Figure A2.

- **Registration (`CLIENT_HELLO`):** The client transmits session name, a resume cursor `(last_segment, last_revision)` (set to `(0, 0)` on initial join), and requested recovery window `history_seconds` (clamped to [0, 60s]).
- **Catch-Up Snapshot (`HISTORY`):** The server returns an atomic, chronologically ordered array of finalized captions strictly newer than the resume cursor within the requested history window. Active provisional drafts are excluded from `HISTORY` and pushed as distinct updates immediately after.
- **Live Streaming (`CAPTION_UPDATE`):** Transmits segment index, revision index, boolean finality flag, text payload, and source timestamp.

2 Revision and Recovery Semantics

Monotonic Ingestion Invariants

Automatic Speech Recognition decodes audio through progressive acoustic revisions before emitting a finalized segment. The server enforces three strict invariants in `accept_caption`:

1. **Monotonic Revisions:** Stale revisions ($R \leq R_{Current}$) for an active segment are dropped immediately.
2. **Finality Seal:** Once a segment is marked $F = True$, all subsequent updates for that segment are rejected.
3. **Monotonic Segments:** A finalized segment permanently seals that index, and the session cursor never moves backwards.

60-Second Ring Buffer & Incremental Recovery

- **Bounded History:** Finalized segments are retained in a per-session ring buffer. Every connection and live update triggers `prune_history(now)`, evicting items older than 60.0s. At speech rates (≈ 1.25 segments/sec), history is naturally bounded to ≤ 80 entries (≈ 15 KB RAM).
- **Deduplicated Reconnect:** Reconnecting clients supply their highest confirmed final cursor. The server transmits strictly missing segments (S > last_segment), achieving sub-3ms recovery (P95 = 2.43 ms) with zero duplicate or stale finals.

- **UI Recall Decoupling:** In the browser client (`client/app.js`), the "What did I miss?" recall feature (15s/30s/60s) operates purely against client-side memory. It does not overwrite the WebSocket connection resume cursor, preventing duplicate streaming loops.

3 Backpressure/Overload Policy

The Kernel Socket Buffer Blind Spot

In `asyncio`, `await ws.send()` returns immediately once data is buffered in the kernel TCP send buffer (`SO_SNDBUF`, ≈ 128 KB). For 150-byte caption frames, over 600 frames can buffer in kernel space before Python encounters socket backpressure. Under slow consumption, frames become obsolete while stuck in kernel memory, causing extreme tail latencies (P99 > 24 sec) and final drops.

Ping/Pong Sliding Flow Window (`FLOW_WINDOW = 4`)

LiveRecall uses RFC 6455 WebSocket control frames to implement application-transparent flow control without client protocol changes (see Figure A3):

- After every message, `client_sender` triggers `client.ws.ping()`.
- According to Section 5.5 of RFC 6455, WebSocket peers must process control frames in order and echo payloads in Pong responses. Each pong acts as a cumulative transport-level ACK.
- The server limits in-flight messages: `client.sent - client.acked < FLOW_WINDOW`, where `FLOW_WINDOW = 4`.
- When the window fills, the sender suspends (`window_open.wait()`). Backpressure is forced back into Python user-space memory, enabling semantic coalescing.

Three-Tier `CoalescingQueue`

Every client is allocated an isolated queue governed by a three-tier policy (see Figure A4):

Tier 1 (Normal FIFO): When unconstrained (`depth < 15`, `head_age < 250ms`, `send_ema < 50ms`, `flow_blocked == False`), all provisional revisions are queued in strict FIFO order, preserving 100% revision coverage.

Tier 2 (In-Place Coalescing): Under strain, the queue switches to semantics-aware coalescing. If a provisional revision arrives for a segment already queued, `_provisional_map[segment]` updates the payload in-place. Obsolete intermediate drafts are crushed without generating extra frames. Final captions are never coalesced or overwritten.

Tier 3 (Capacity Protection): If queue depth exceeds `HARD_QUEUE_LIMIT = 128`, oldest unfinalized provisional items are evicted. If 128 final captions accumulate (frozen reader), the server terminates the connection with code `1013` ("Try Again Later") to preserve server stability.

Multi-Tier Reader Classification & Degradation

1. **RTT-Relative Slow Mode (`10 × min_rtt`):** To prevent false positives on high-latency networks (e.g. T5 with 120ms netem delay), the server tracks `min_rtt`. A pong triggers `slow_mode` only if latency exceeds 10×min_rtt and exceeds 250 ms (`SLOW_MIN_LATENCY`).

2. **Bandwidth Freshness Trigger (`STALE_PONG_STREAK = 3`):** Under severe bandwidth starvation (300 clients on 2 Mbit/s), initial baseline RTT is inflated. If 3 consecutive non-overlapping pongs exceed 500 ms (`FRESHNESS_LIMIT`), the client enters `slow_mode`.
3. **Finals-Only Delivery:** In `slow_mode`, provisional updates are dropped at ingest (`if not is_final and client.slow_mode: continue`). Bandwidth consumption drops by 96%, allowing constrained clients to receive durable final captions at ≈1.25 msgs/sec with near-zero latency.

4 Public Tests

We ran the public test scenarios from the assignment harness against `solution_server.py` on the loopback interface (`lo`). `run_demo.py` starts the server, the caption source and the load clients, and `analyse_results.py` computes the metrics. Every scenario was run three times, and each figure we report is the median of the three runs.

The scenarios are:

- **T1:** 20 clients for 20s, no impairment.
- **T2:** 100 clients for 40s, no impairment.
- **T3:** 100 clients for 60s, with 10% of clients reading slowly (a 500ms delay per message).
- **T4:** 100 clients for 60s. 20% of clients disconnect at 15s for 5s and then reconnect.
- **T5:** 100 clients for 40s over a delay and packet-loss profile (`network_profiles/delay_loss.sh`).
- **Bandwidth (BW300):** 300 clients for 40s on a link capped at 2 Mbit/s (`network_profiles/bandwidth_2mbit.sh`).
- **Burst 20 / Burst 100:** 20 clients for 20s (22 segments) and 100 clients for 40 s (49 segments), with revisions sent back to back (`--revision-delay 0`) and a 0.75s gap between segments.

The commands shown in Figure A5 reproduce every run. The network profiles need sudo and are reset after each impaired scenario.

4.1 Before/After Optimization Comparison

Optimization 1: Ping/Pong Flow Window

Table A1 compares the server before and after optimization 1. The main gain is in T3: completeness rose from 95.13% to 98.68%, slow-client P50 fell from 14.71s to 8.08s, and the overall P99 fell from 24.02s to 8.82s. Latency fell because a slow client's messages now wait in the server's queue, where stale revisions are coalesced, instead of piling up in the kernel send buffer behind hundreds of obsolete frames. T3 completeness was still below the 99% gate. In the unimpaired tests latency rose by a few milliseconds (T2 P95 from 4.5ms to 9.8ms), well below the 500ms freshness threshold, and T5 coverage fell slightly from 99.99% to 99.20%. The small rise in latency comes from the window itself: every message is now followed by a ping, and once four messages are unconfirmed the server waits for a pong before sending more.

Optimization 2: RTT-Relative Slow Mode

Table A2 compares the server before and after optimization 2. In T3, completeness rose from 98.76% to 99.21%, passing the 99% gate, slow-client P50 fell from 8.14s to 3.62s, and the overall P95 fell from 2.28s to 25.9ms. Both drops come from slow clients receiving finals only: they no longer read

through a backlog of provisionals, and their messages fall to about 5% of all T3 messages, so the overall P95 is now set by normal clients rather than slow ones. The cost is T3 coverage, which fell from 95.18% to 94.91%. Coverage and completeness in T1, T2 and both burst tests stay at 100%. The small latency changes in T1, T2 and the burst tests (a few milliseconds in either direction) are run-to-run variation, since no client enters slow mode in those tests.

Optimization 3: Bandwidth Freshness Trigger

Table A3 compares the server before and after optimization 3. In the bandwidth test (300 clients on 2 Mbit/s), the fresh final ratio rose from 0.00% to 87.24%, the fresh caption ratio from 0.90% to 78.92% and completeness from 98.93% to 100%, while P50 fell from 1.14s to 82ms. P50 fell because clients now switch to finals only after three stale pongs, which cuts their traffic enough for the 2 Mbit/s link to carry finals without a long queue. P99 is still above the threshold (1.27s), most likely from messages sent before a client has collected its three stale pongs. The cost is coverage in this test, which fell from 85.77% to 37.44% (Section 5). The public tests show no regression: T3 is unchanged, completeness stays at 100% in T1, T2, T4, T5 and both burst tests, and T5 coverage moved only from 99.36% to 99.15%. T5 P95 also fell from 642ms to 522ms, which, together with the small drop in coverage, suggests the trigger occasionally fires there when delay and loss push several consecutive pongs past 500ms.

4.2 P50/P95/P99/Freshness/Final-Completeness Results

Table A4 shows the final server, with all three optimizations, on every public scenario. Latency is measured per caption message and pooled over all clients. The fresh ratios count messages delivered within the 500ms freshness threshold, completeness is the share of expected final captions received, and coverage is the share of expected revisions received. No run produced a duplicate or stale final, and in T4 reconnecting clients recovered their missed finals with a recovery P95 of 2.4ms.

Coverage is below 100% only where slow mode is active (T3 and BW300) or where clients were disconnected (T4, since HISTORY replays finals but not provisional drafts). Section 5 discusses this trade-off.

5 Failures and Trade-offs

Slow Mode: Coverage vs. Completeness and Freshness

Slow mode trades revision coverage for final completeness and freshness. A client in slow mode receives finals only, so every provisional revision it skips counts against coverage. In exchange, its limited read capacity goes to the finals, which count toward completeness and stay fresh instead of queuing behind drafts that will be replaced anyway.

The measurements show both sides of the trade:

- **T3 (optimization 2):** completeness rose from 98.76% to 99.21%, passing the 99% gate, and slow-client P50 fell from 8.14s to 3.62s. Coverage fell from 95.18% to 94.91%.
- **BW300 (optimization 3):** the fresh final ratio rose from 0.00% to 87.24%, the fresh caption ratio from 0.90% to 78.92% and completeness from 98.93% to 100%, while P50 fell from 1.14s to 82ms. Coverage fell from 85.77% to 37.44%.

We accept the lower coverage because the two losses are not equal. A skipped provisional is a draft that a later revision or the final replaces, while a missing or stale final is a lasting gap in the transcript.

The BW300 result also shows that high coverage alone does not help the reader: before optimization 3 the server delivered 85.77% of revisions, but almost none arrived within 500ms, so students were reading captions more than a second old. Under normal conditions slow mode never triggers, and coverage stays at 100% in T1, T2 and both burst tests. The 250ms floor and the 10 × `min_rtt` rule also keep high-latency clients that are still keeping up out of slow mode, so T5 keeps 99.15% coverage despite its added delay.

The same trade-off sets the exit rule. A client leaves slow mode after 50 confirmed messages with no slow pong (`SLOW_EXIT_MESSAGES = 50`). Since a client in slow mode receives about 1.25 finals/sec, this takes roughly 40s, so a healthy client that hits one transient spike over 250ms loses provisionals for that long. We tested a threshold of 6, which would let such a client recover in about 5s, but genuinely slow clients then fell back to normal mode many times and T3 completeness dropped below 99%. We kept 50, preferring completeness on genuinely slow clients over coverage on briefly delayed ones. An exit threshold that adapts to each client's behaviour would reduce this cost, and we leave it as future work.

6 Limitations

LiveRecall's single-process Python architecture limits its scalability. Although asyncio supports non-blocking I/O for hundreds of clients, CPU-bound operations such as JSON serialization (`json.dumps`) can delay other tasks on the same event loop as connection counts increase. The precise capacity depends on the hardware, message rate and payload size.

Another limitation is that caption updates replace entire segments rather than transmit character-level differences. This simplifies revision handling but can increase bandwidth consumption, particularly when successive revisions contain only minor changes. Under severe bandwidth constraints, repeatedly transmitting full strings may be less efficient than differential updates.

Lastly, WebSockets rely on TCP and are therefore subject to head-of-line blocking. When a TCP segment is lost, subsequent data cannot be delivered to the application until the missing data is recovered. This can introduce latency spikes under packet loss, including the conditions represented by the T5 test, even when application-level queue management is effective.

7 Future-work/FYP Direction

Future work could explore migrating from WebSockets to WebTransport over HTTP/3 and QUIC. Independent streams could reduce head-of-line blocking between streams, although reliable delivery within each stream would still require recovery of lost data.

Another improvement would make `SLOW_EXIT_MESSAGES` dynamic. The server could adjust the threshold using recent queue depth, acknowledgement timing and transmission performance, allowing faster recovery from slow mode while limiting frequent switching between modes.

A longer-term FYP direction could investigate predictive backpressure through Kalman filtering or reinforcement learning to adjust `FLOW_WINDOW` for each client. However, Pong timing indicates network and protocol responsiveness rather than directly measuring rendering capacity, so explicit client-side measurements could strengthen predictions. Evaluation against the existing threshold-based policy should assess latency, caption completeness and computational overhead to determine whether the additional complexity provides meaningful benefits.

Appendix

Figure A1

LiveRecall System Architecture and Single-Authoritative-Server Topology

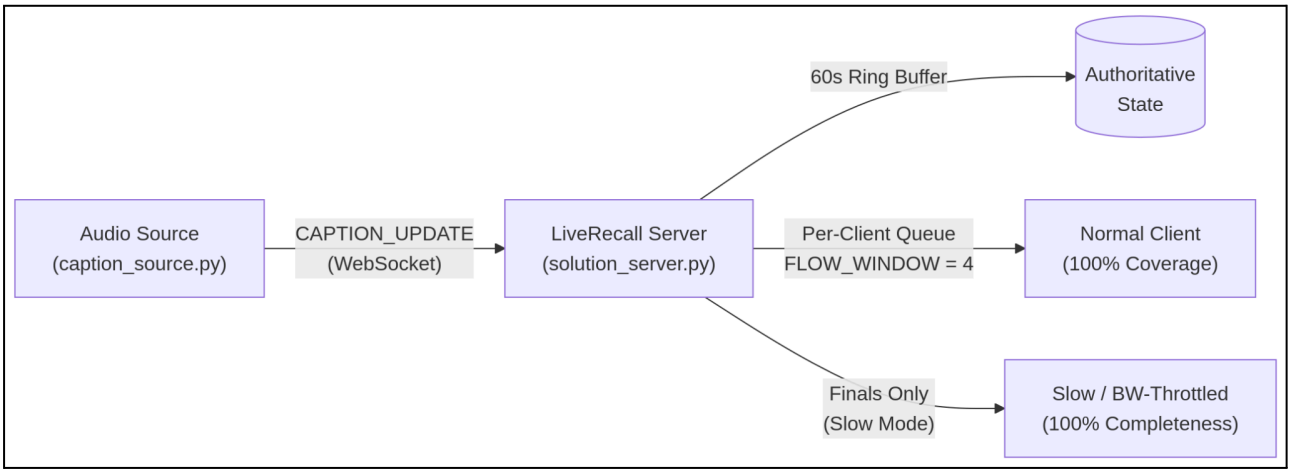


Figure A2

LiveRecall Protocol Message Exchange Sequence

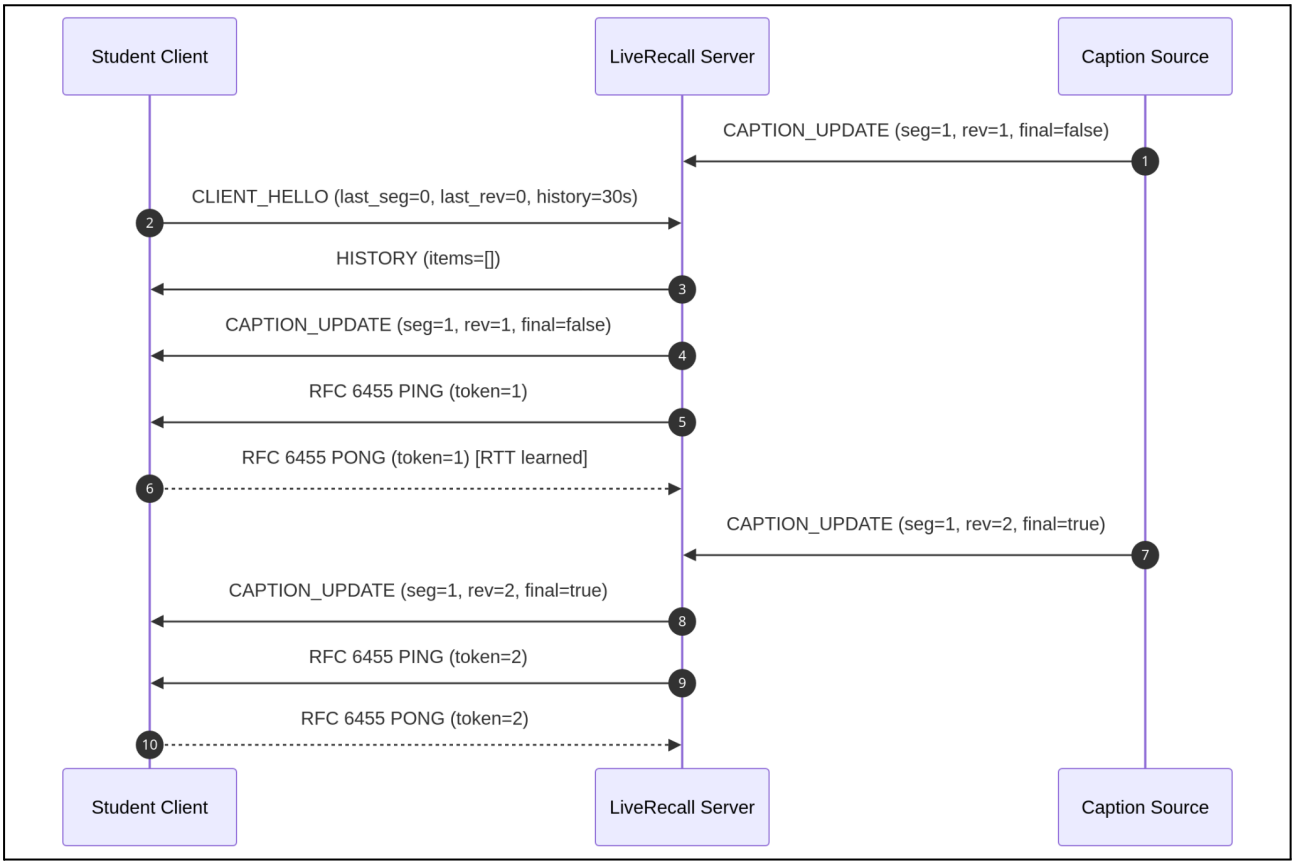


Figure A3

LiveRecall Sliding Flow Window and Backpressure State Transitions

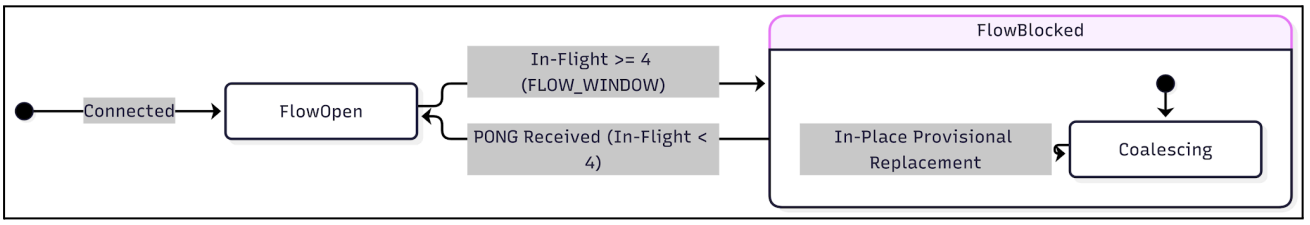


Figure A4

Three-Tier CoalescingQueue Processing and Capacity Protection

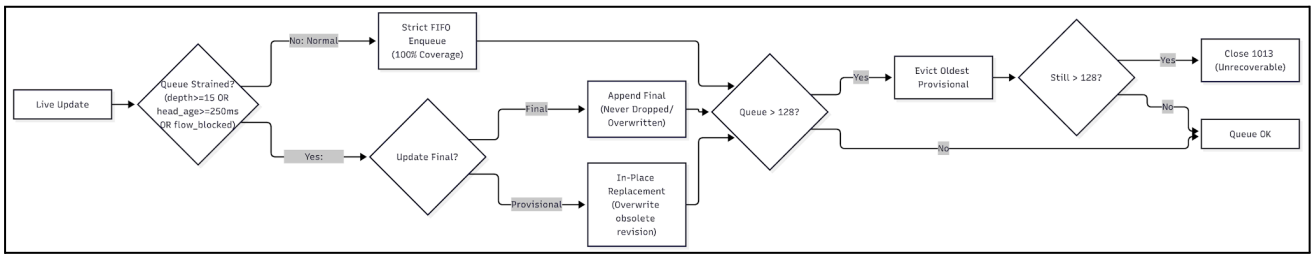


Figure A5

Commands for LiveRecall Performance Testing and Results Analysis

```
# Bandwidth
sudo ./network_profiles/bandwidth_2mbit.sh lo
python run_demo.py --clients 300 --duration 40 --runs 3 --output-stem bw300
sudo ./network_profiles/reset.sh lo

# T1-T4
python run_demo.py --clients 20 --duration 20 --runs 3 --output-stem t1
python run_demo.py --clients 100 --duration 40 --runs 3 --output-stem t2
python run_demo.py --clients 100 --duration 60 --runs 3 --slow-percent 10 --slow-ms 500
    --output-stem t3
python run_demo.py --clients 100 --duration 60 --runs 3 --disconnect-percent 20 --disconnect-at 15
    --disconnect-seconds 5 --output-stem t4

# T5
sudo ./network_profiles/delay_loss.sh lo
python run_demo.py --clients 100 --duration 40 --runs 3 --output-stem t5
sudo ./network_profiles/reset.sh lo

# Burst (N clients, D seconds, S segments: 20/20/22 and 100/40/49), once per run
python solution_server.py --port 8765 &
python load_clients.py --uri ws://127.0.0.1:8765 --clients N --duration D --output burstN.csv --seed
    3103 --history-seconds 30 &
sleep 1
python caption_source.py --uri ws://127.0.0.1:8765 --segments S --revision-delay 0 --segment-gap
    0.75 --quiet
wait %2; kill %1
python analyse_results.py burstN.csv --expected-finals S --expected-updates $((S * 3)) --json-output
    burstN.json
```

Table A1

Optimization 1 before and after (median of three runs)

Test	Metric	Before	After (flow window)
Burst 20 / 100	Coverage	100% / 100%	100% / 100%
	P99	4.0 / 15.6 ms	4.8 / 18.1 ms
T1	Coverage / completeness	100% / 100%	100% / 100%
	P99	1.6 ms	2.5 ms
T2	Coverage / completeness	100% / 100%	100% / 100%
	P95 / P99	4.5 / 7.8 ms	9.8 / 11.7 ms
T3	Completeness	95.13%	98.68%
	Slow-client P50 / overall P99	14.71 s / 24.02 s	8.08 s / 8.82 s
	Normal-client P95	7.5 ms	10.1 ms
T4	Completeness / recovery P95	100% / 3.5 ms	100% / 3.8 ms
	P95	6.6 ms	14.5 ms
T5	Coverage	99.99%	99.20%
	Completeness	100%	100%
	P99 / fresh final ratio	940 ms / 89.4%	994 ms / 86.5%

Table A2

Optimization 2 before and after (median of three runs)

Test	Metric	Before	After (RTT slow mode)
Burst 20 / 100	Coverage	100% / 100%	100% / 100%
	P99	10.3 / 33.9 ms	13.6 / 47.4 ms
T1	Coverage / completeness	100% / 100%	100% / 100%
	P99	7.6 ms	10.0 ms
T2	Coverage / completeness	100% / 100%	100% / 100%
	P95 / P99	24.8 / 32.5 ms	18.8 / 27.9 ms
T3	Completeness	98.76%	99.21%
	Slow-client P50 / overall P99	8.14 s / 8.85 s	3.62 s / 6.60 s
	Overall P95	2.28 s	25.9 ms

	Fresh final ratio	90.00%	91.97%
	Coverage	95.18%	94.91%
	Normal-client P95	18.2 ms	16.4 ms
T4	Completeness / recovery P95	100% / 5.3 ms	100% / 8.1 ms
	P95	16.0 ms	19.7 ms
T5	Coverage	99.12%	99.24%
	Completeness	100%	100% (one run 99.96%)
	P99 / fresh final ratio	992 ms / 86.3%	1,069 ms / 87.6%

Table A3

Optimization 3 before and after (median of three runs)

Test	**Metric**	**Before**	**After (freshness trigger)**
BW300	Fresh final ratio	0.00%	87.24%
	Completeness	98.93%	100%
	P50 / P99	1.14 s / 1.72 s	82 ms / 1.27 s
	Fresh caption ratio	0.90%	78.92%
	Coverage	85.77%	37.44%
Burst 20 / 100	Coverage	100% / 100%	100% / 100%
	P99	5.5 / 27.5 ms	3.1 / 39.3 ms
T1	Coverage / completeness	100% / 100%	100% / 100%
	P99	4.9 ms	4.7 ms
T2	Coverage / completeness	100% / 100%	100% / 100%
	P95 / P99	11.2 / 18.1 ms	14.9 / 20.0 ms
T3	Completeness / coverage	99.21% / 94.91%	99.21% / 94.91%
	Slow-client P50 / overall P99	3.59 s / 6.60 s	3.59 s / 6.59 s
	Normal-client P95	11.2 ms	8.5 ms
T4	Completeness / recovery P95	100% / 2.2 ms	100% / 2.4 ms
	P95	5.3 ms	5.3 ms
T5	Coverage / completeness	99.36% / 100%	99.15% / 100%

	P95 / P99	642 / 813 ms	522 / 749 ms
	Fresh final ratio	89.71%	92.20%

Table A4

Final server on all public scenarios (median of three runs)

Test	P50	P95	P99	Fresh captions	Fresh finals	Completeness	Coverage
T1	2.29 ms	2.97 ms	4.72 ms	100%	100%	100%	100%
T2	6.74 ms	14.93 ms	19.98 ms	100%	100%	100%	100%
T3	5.90 ms	12.32 ms	6.59 s	95.70%	91.97%	99.21%	94.91%
T4	3.31 ms	5.29 ms	9.89 ms	100%	98.42%	100%	98.25%
T5	253 ms	522 ms	749 ms	94.40%	92.20%	100%	99.15%
BW300	82 ms	984 ms	1.27 s	78.92%	87.24%	100%	37.44%
Burst 20	2.04 ms	2.88 ms	3.12 ms	100%	100%	100%	100%
Burst 100	11.97 ms	26.14 ms	39.27 ms	100%	100%	100%	100%